



New Mansoura University
Faculty of Computer Science and Engineering



Advanced Machine Learning

Course Description

Islam Reda

Associate Professor

islam.reda@nmu.edu.eg

Course Description

Covers supervised and unsupervised learning.

- Starts with an end-to-end machine learning project.
- Introduces regularization techniques.
- Explores **ensemble learning**: Bagging, Boosting, Stacking.
- Includes **unsupervised learning**: K-means, DBSCAN and Gaussian mixture clustering.
- Introduces **dimensionality reduction**: PCA

Learning Outcomes

By the end of the course, students will be able to:

- Apply advanced supervised learning algorithms and Implement ensemble learning methods to improve model performance.
- Apply different regularization methods to overcome the overfitting problem.
- Analyze and cluster unlabeled datasets using unsupervised learning.
- Apply dimensionality reduction techniques for data visualization and preprocessing.

References

- Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow Concepts, Tools, and Techniques to Build Intelligent Systems
- Machine Learning A-Z





New Mansoura University
Faculty of Computer Science and Engineering



Advanced Machine Learning

End-to-End Machine Learning Project

Islam Reda

Associate Professor

islam.reda@nmu.edu.eg

End-to-End Machine Learning Project

1. Look at the big picture.
2. Get the data.
3. Discover and visualize the data to gain insights.
4. Prepare the data for Machine Learning algorithms.
5. Select a model and train it.
6. Fine-tune your model.
7. Launch, monitor, and maintain your system.

Working with Real Data

When you are learning about Machine Learning it is best to actually experiment with real-world data. Here are a few places you can look to get data:

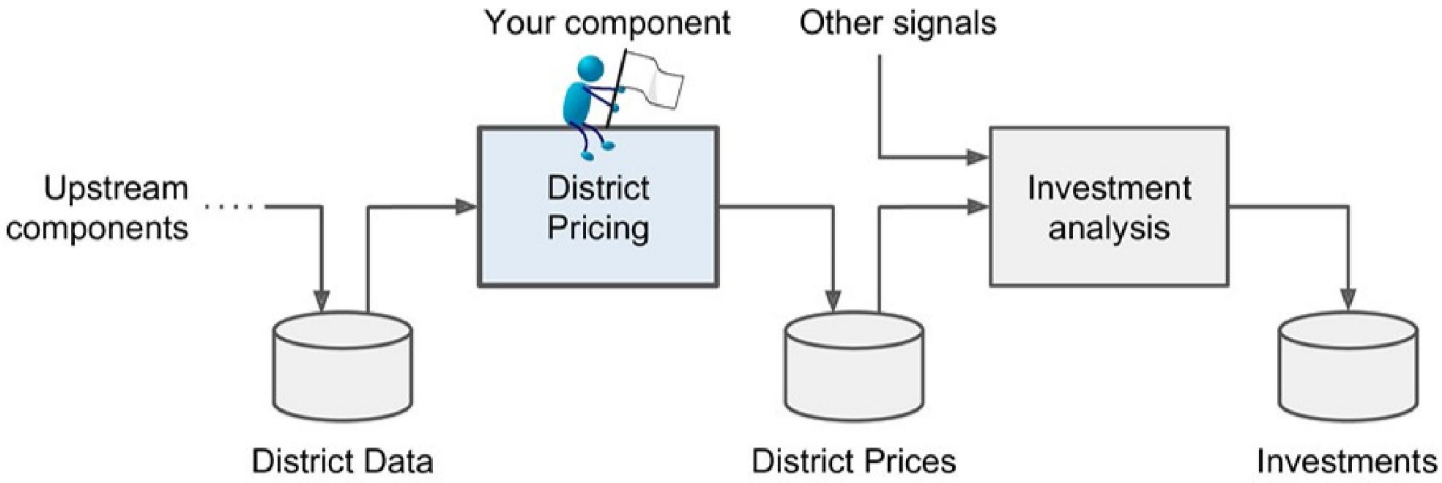
- Popular open data repositories:
  - UC Irvine Machine Learning Repository
  - Kaggle datasets
  - Amazon's AWS datasets
- In this lecture we chose the California Housing Prices dataset from the StatLib repository.

1- Look at the Big Picture

1- Look at the Big Picture

- The first task you are asked to perform is to build a model of housing prices in California using the California census data.
- Your model should learn from this data and be able to predict the median housing price in any district.
- **Frame the Problem**:
  - The first question to ask is what exactly is the business objective; How does the company expect to use and benefit from this model?

1- Look at the Big Picture



A Machine Learning pipeline for real estate investments

- **Frame the Problem:**
  - Is it supervised, unsupervised, or Reinforcement Learning? Is it a classification task, a regression task, or something else? Should you use batch learning or online learning techniques?

1- Look at the Big Picture

- **Select a Performance Measure**:
  - A typical performance measure for regression problems is the Root Mean Square Error (RMSE) (Euclidean norm - l_2 norm).

$$\mathrm{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(h(\mathrm{x}^{(i)} - y^{(i)})^2}$$

  - If there are many outlier districts. In that case, you may consider using the Mean Absolute Error (MAE) (Manhattan norm - l_1 norm).

$$\mathrm{MAE}(\mathbf{X}, h) = \frac{1}{m}\sum_{i=1}^{m}\left|h(\mathrm{x}^{(i)} - y^{(i)}\right|$$

2- Get the Data

2- Get the Data

- **Creating an Isolated Environment**:
  - If you would like to work in an isolated environment (which is strongly recommended so you can work on different projects without having conflicting library versions), create a virtual environment.
  - Open Command Prompt or PowerShell
  - Navigate to Your Project Folder

    mkdir my_project

    cd my_project
  - Create the Virtual Environment
    - Use Python's built-in venv module to create a virtual environment:

    python -m venv venv
  - Install Packages
  - Activate the Virtual Environment

    venv\Scripts\activate

2- Get the Data

- **Download the Data:**
  - load the data using Pandas:

```
import pandas as pd
def load_housing_data(housing_path=HOUSING_PATH):
    csv_path = os.path.join(housing_path, "housing.csv")
    return pd.read_csv(csv_path)
```

- **Take a Quick Look at the Data Structure**

```
In [5]: housing = load_housing_data()
        housing.head()
```

Out[5]:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	populatio
0	-122.23	37.88	41.0	880.0	129.0	322.0
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0
2	-122.24	37.85	52.0	1467.0	190.0	496.0
3	-122.25	37.85	52.0	1274.0	235.0	558.0
4	-122.25	37.85	52.0	1627.0	280.0	565.0

2- Get the Data

- **Take a Quick Look at the Data Structure**

```
In [6]: housing.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
longitude             20640 non-null float64
latitude              20640 non-null float64
housing_median_age    20640 non-null float64
total_rooms           20640 non-null float64
total_bedrooms        20433 non-null float64
population            20640 non-null float64
households            20640 non-null float64
median_income         20640 non-null float64
median_house_value    20640 non-null float64
ocean_proximity       20640 non-null object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

```
>>> housing["ocean_proximity"].value_counts()
<1H OCEAN     9136
INLAND        6551
NEAR OCEAN    2658
NEAR BAY      2290
ISLAND           5
Name: ocean_proximity, dtype: int64
```

2- Get the Data

- **Take a Quick Look at the Data Structure**

```
In [8]: housing.describe()
```

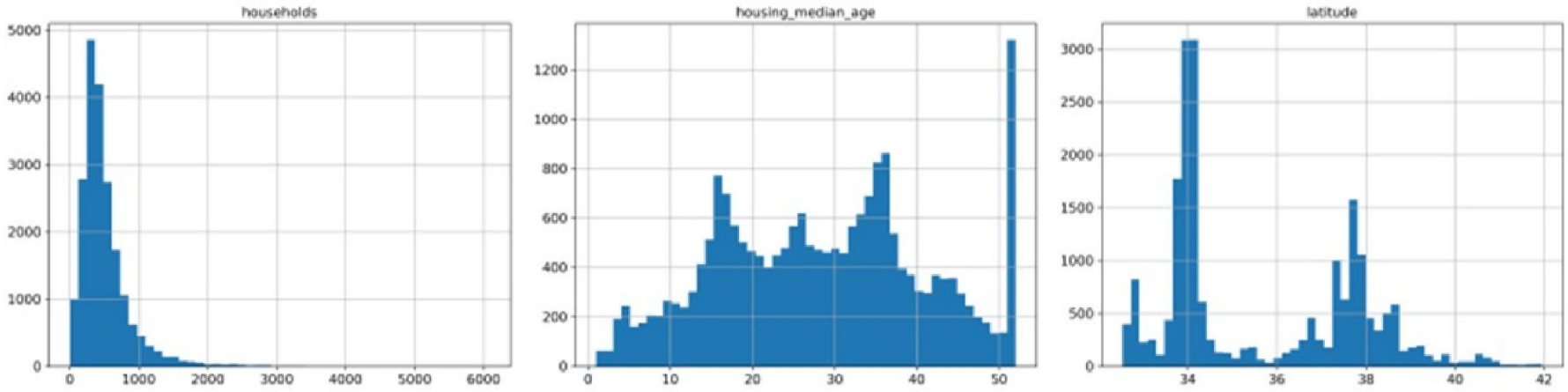
Out[8]:

	longitude	latitude	housing_median_age	total_rooms	total_bedr
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000(
mean	-119.569704	35.631861	28.639486	2635.763081	537.87055:
std	2.003532	2.135952	12.585558	2181.615252	421.38507(
min	-124.350000	32.540000	1.000000	2.000000	1.000000
25%	-121.800000	33.930000	18.000000	1447.750000	296.00000(
50%	-118.490000	34.260000	29.000000	2127.000000	435.00000(
75%	-118.010000	37.710000	37.000000	3148.000000	647.00000(
max	-114.310000	41.950000	52.000000	39320.000000	6445.0000(

2- Get the Data

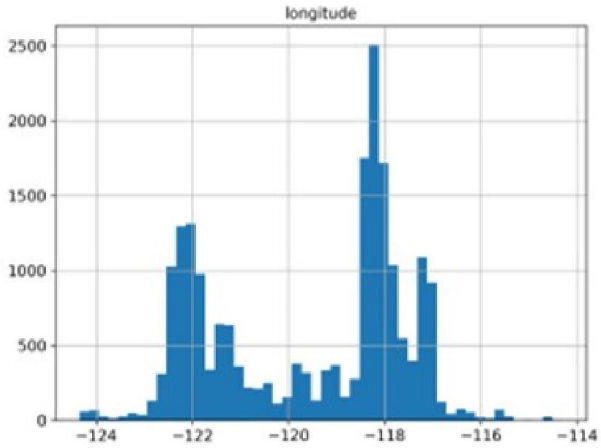
- **Take a Quick Look at the Data Structure**

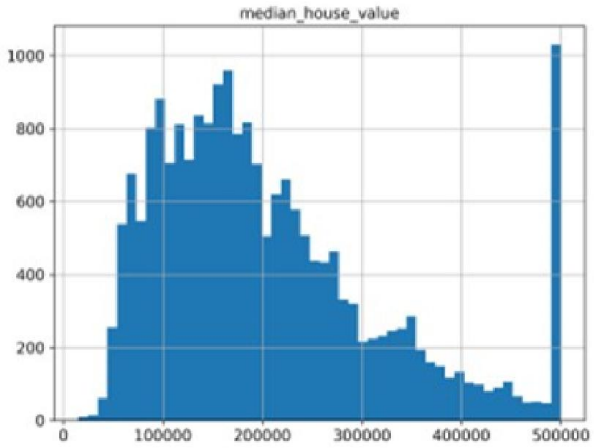
```
%matplotlib inline # only in a Jupyter notebook
import matplotlib.pyplot as plt
housing.hist(bins=50, figsize=(20,15))
plt.show()
```

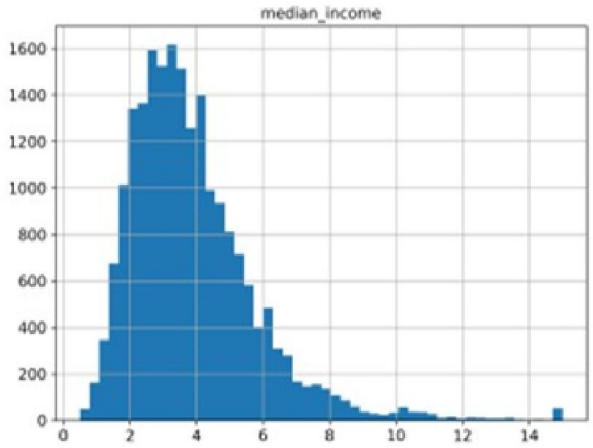


2- Get the Data

- **Take a Quick Look at the Data Structure**







2- Get the Data

- **Create a Test Set**
  - Scikit-Learn provides a few functions to split datasets into multiple subsets in various ways. The simplest function is train_test_split.

```
from sklearn.model_selection import train_test_split

train_set, test_set = train_test_split(housing, test_size=0.2, random_state=42)
```

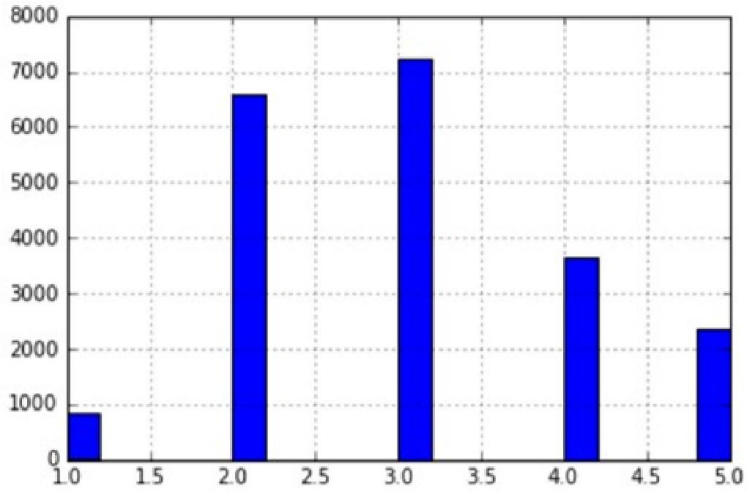
  - So far we have considered purely **random sampling** methods.
  - **Stratified sampling**: the population is divided into homogeneous subgroups called strata, and the right number of instances is sampled from each stratum to guarantee that the test set is representative of the overall population.

2- Get the Data

- **Create a Test Set**
  - The following code uses the pd.cut() function to create an income category attribute with 5 categories (labeled from 1 to 5):

```
housing["income_cat"] = pd.cut(housing["median_income"],
                               bins=[0., 1.5, 3.0, 4.5, 6., np.inf],
                               labels=[1, 2, 3, 4, 5])
```

```
housing["income_cat"].hist()
```



2- Get the Data

- **Create a Test Set**
  - Now you are ready to do stratified sampling based on the income category. For this you can use train_test_split by setting the **stratify** parameter to the income category:

```
from sklearn.model_selection import train_test_split

strat_train_set, strat_test_set = train_test_split(housing, test_size=0.2, random_state=42,
stratify=housing["income_cat"])
```

  - You can look at the income category proportions in the test set:

```
>>> strat_test_set["income_cat"].value_counts() / len(strat_test_set)
3 0.350533
2 0.318798
4 0.176357
5 0.114583
1 0.039729
```

2- Get the Data

- **Create a Test Set**
  - This figure compares the income category proportions in the overall dataset, in the test set generated with stratified sampling, and in a test set generated using purely random sampling.

	Overall	Random	Stratified	Rand. %error	Strat. %error
1.0	0.039826	0.040213	0.039738	0.973236	-0.219137
2.0	0.318847	0.324370	0.318876	1.732260	0.009032
3.0	0.350581	0.358527	0.350618	2.266446	0.010408
4.0	0.176308	0.167393	0.176399	-5.056334	0.051717
5.0	0.114438	0.109496	0.114369	-4.318374	-0.060464

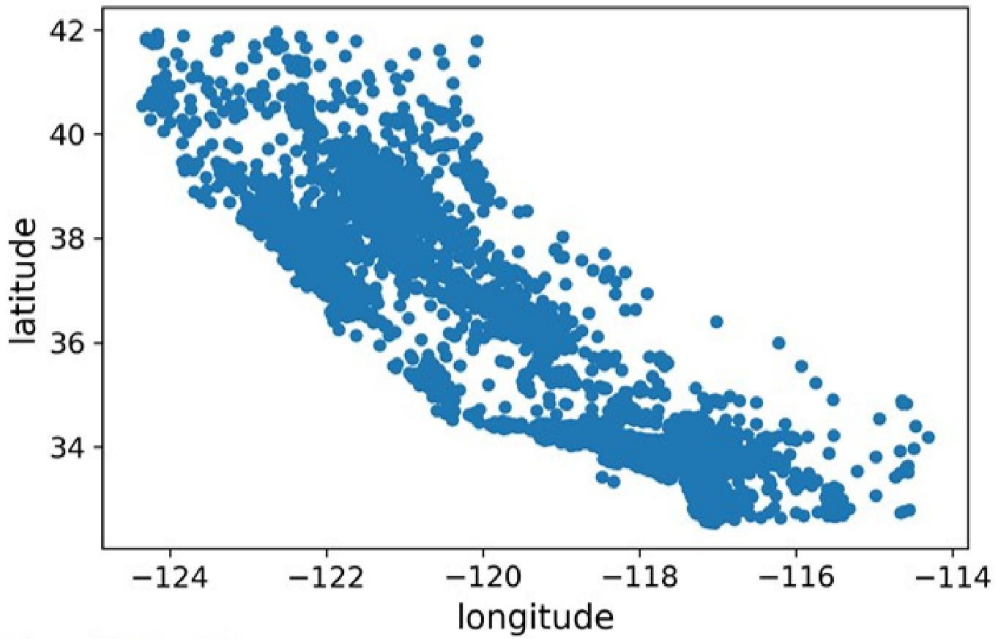
3- Discover and Visualize the Data to Gain Insights

3- Discover and Visualize the Data to Gain Insights

- Let's create a copy so you can play with it without harming the training set:

```
housing = strat_train_set.copy()
```

- **Visualizing Geographical Data**
  - Since there is geographical information (latitude and longitude), it is a good idea to create a scatterplot of all districts to visualize the data.

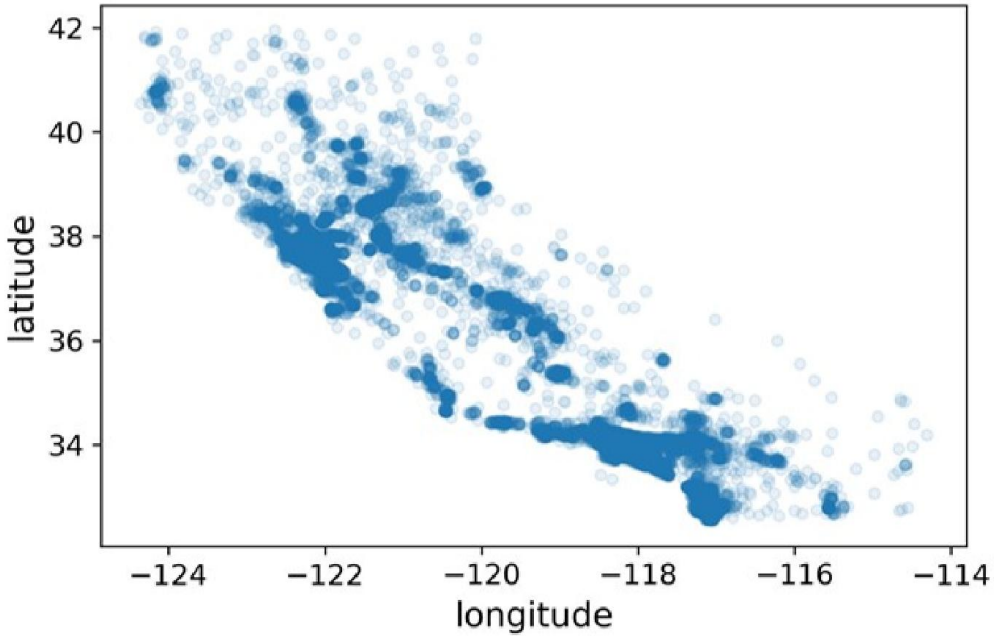



```
housing.plot(kind="scatter", x="longitude", y="latitude")
```

3- Discover and Visualize the Data to Gain Insights

- **Visualizing Geographical Data**
  - It is hard to see any particular pattern. Setting the alpha option to 0.1 makes it much easier to visualize the places where there is a high density of data points

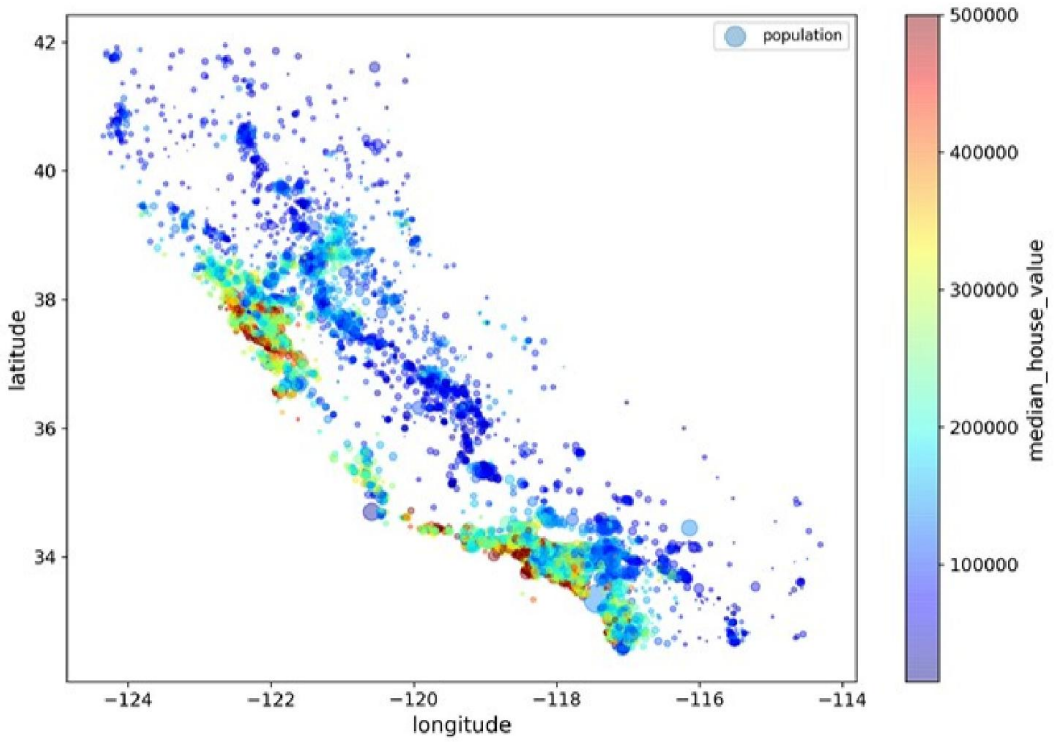
```
housing.plot(kind="scatter", x="longitude",
y="latitude", alpha=0.1)
```



3- Discover and Visualize the Data to Gain Insights

- **Visualizing Geographical Data**

```
housing.plot(kind="scatter",
x="longitude", y="latitude", alpha=0.4,
s=housing["population"]/100,
label="population", figsize=(10,7),
c="median_house_value",
cmap=plt.get_cmap("jet"),
colorbar=True)
plt.legend()
```



3- Discover and Visualize the Data to Gain Insights

- **Looking for Correlations**
  - You can compute the standard correlation coefficient (also called Pearson's r) between every pair of attributes using the corr() method:

```
corr_matrix = housing.corr()
```

  - Now let's look at how much each attribute correlates with the median house value:

```
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
```

```
median_house_value    1.000000
median_income         0.687170
total_rooms           0.135231
housing_median_age    0.114220
households            0.064702
total_bedrooms        0.047865
population           -0.026699
longitude            -0.047279
latitude             -0.142826
Name: median_house_value, dtype: float64
```

3- Discover and Visualize the Data to Gain Insights

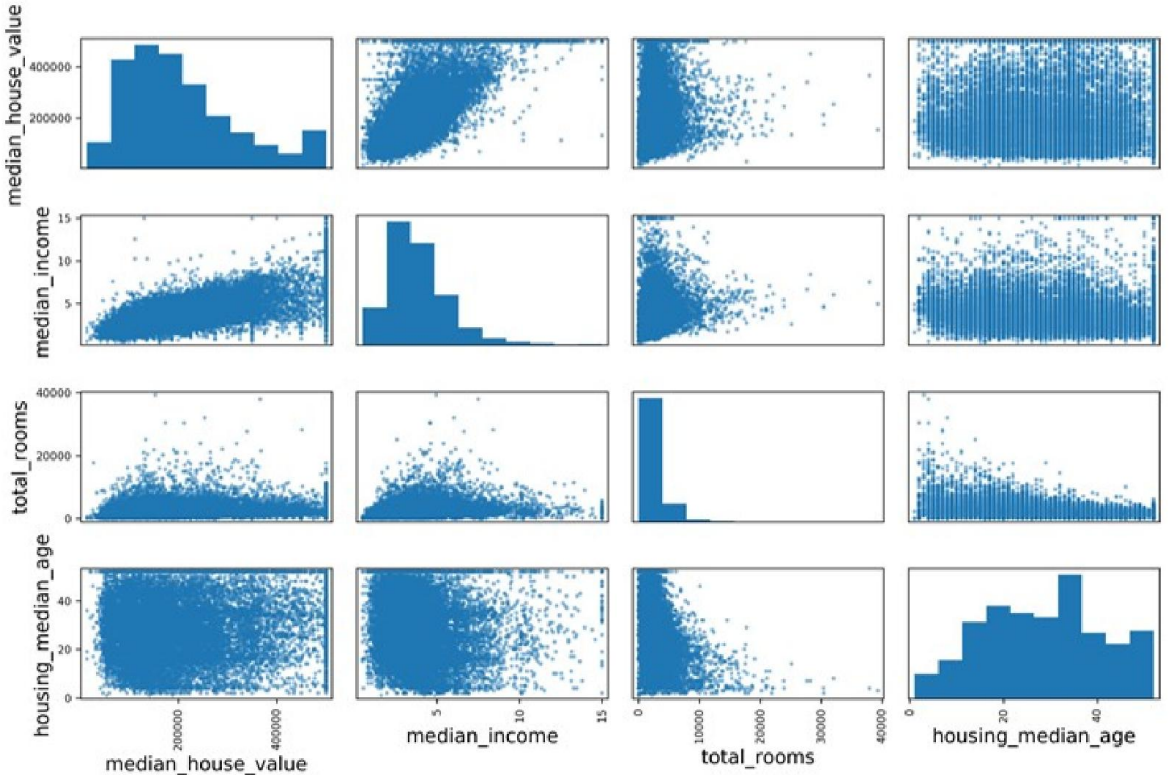
- **Looking for Correlations**
  - Pandas' scatter_matrix function, which plots every numerical attribute against every other numerical attribute.

```
from pandas.plotting import scatter_matrix

attributes = ["median_house_value", "median_income", "total_rooms","housing_median_age"]
scatter_matrix(housing[attributes], figsize=(12, 8))
```

3- Discover and Visualize the Data to Gain Insights

- **Looking for Correlations**

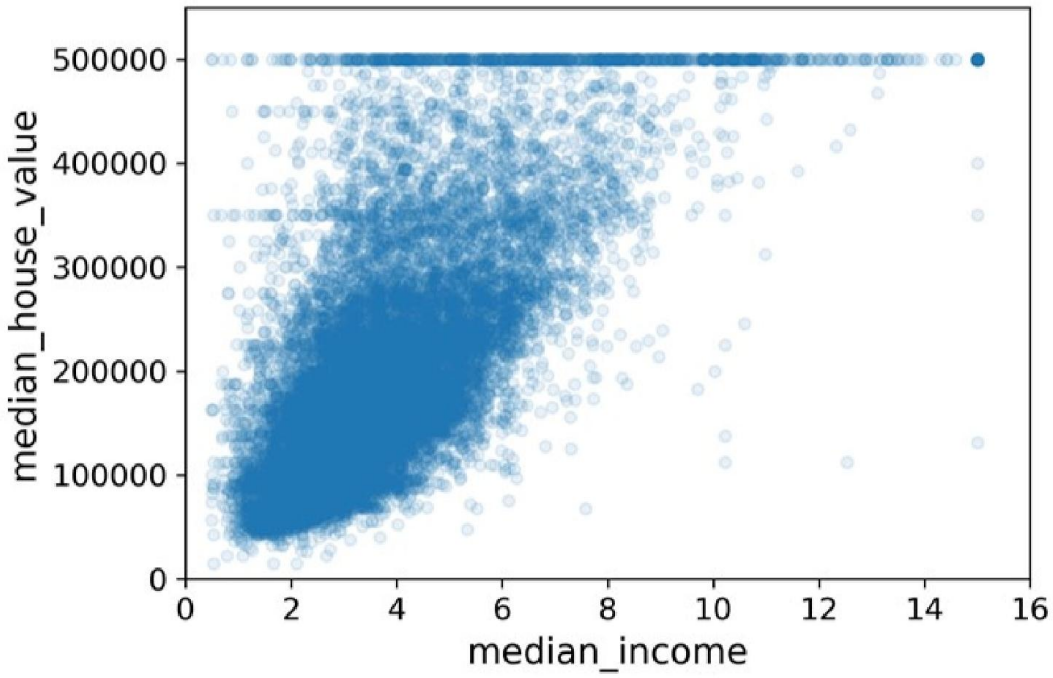


3- Discover and Visualize the Data to Gain Insights

- **Looking for Correlations**

```
housing.plot(kind="scatter",
x="median_income",
y="median_house_value", alpha=0.1)
```

  - The correlation is indeed very strong.
  - The price cap that we noticed earlier is clearly visible as a horizontal line at $500,000.



3- Discover and Visualize the Data to Gain Insights

- **Experimenting with Attribute Combinations**
  - Try out various attribute combinations. For example, the total number of rooms in a district is not very useful if you don't know how many households there are.
  - The total number of bedrooms by itself is not very useful: you probably want to compare it to the number of rooms.
  - The population per household also seems like an interesting attribute

```
housing["rooms_per_household"] = housing["total_rooms"]/housing["households"]
housing["bedrooms_per_room"] = housing["total_bedrooms"]/housing["total_rooms"]
housing["population_per_household"]=housing["population"]/housing["households"]
```

3- Discover and Visualize the Data to Gain Insights

- **Experimenting with Attribute Combinations**
  - Now let's look at the correlation matrix again:

```
>>> corr_matrix = housing.corr()
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
```

```
median_house_value          1.000000
median_income               0.687160
rooms_per_household         0.146285
total_rooms                 0.135097
housing_median_age          0.114110
households                  0.064506
total_bedrooms              0.047689
population_per_household   -0.021985
population                 -0.026920
longitude                  -0.047432
latitude                   -0.142724
bedrooms_per_room          -0.259984
Name: median_house_value, dtype: float64
```

Thank You